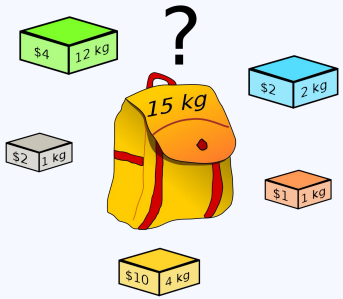
Language used: Java

SC2001 Example Class 3
Dynamic Programming

Groupmates:
Florian
Min Jie
Yi Rou
Yi Ken

Problem given



0/1 Unbounded Knapsack Problem

- Knapsack has capacity C ($C \in \mathbb{Z}^+$)
- Types of objects: n (**Unbounded:** unlimited supplies of each object)
- Each object of ith type has weight w_i and profit p_i, ($i \in [0, n-1]$, $i \in \mathbb{Z}$)

Challenge:

Find *largest total profit* of any set of the objects that fits in the knapsack

Recursive Definition for P(C)

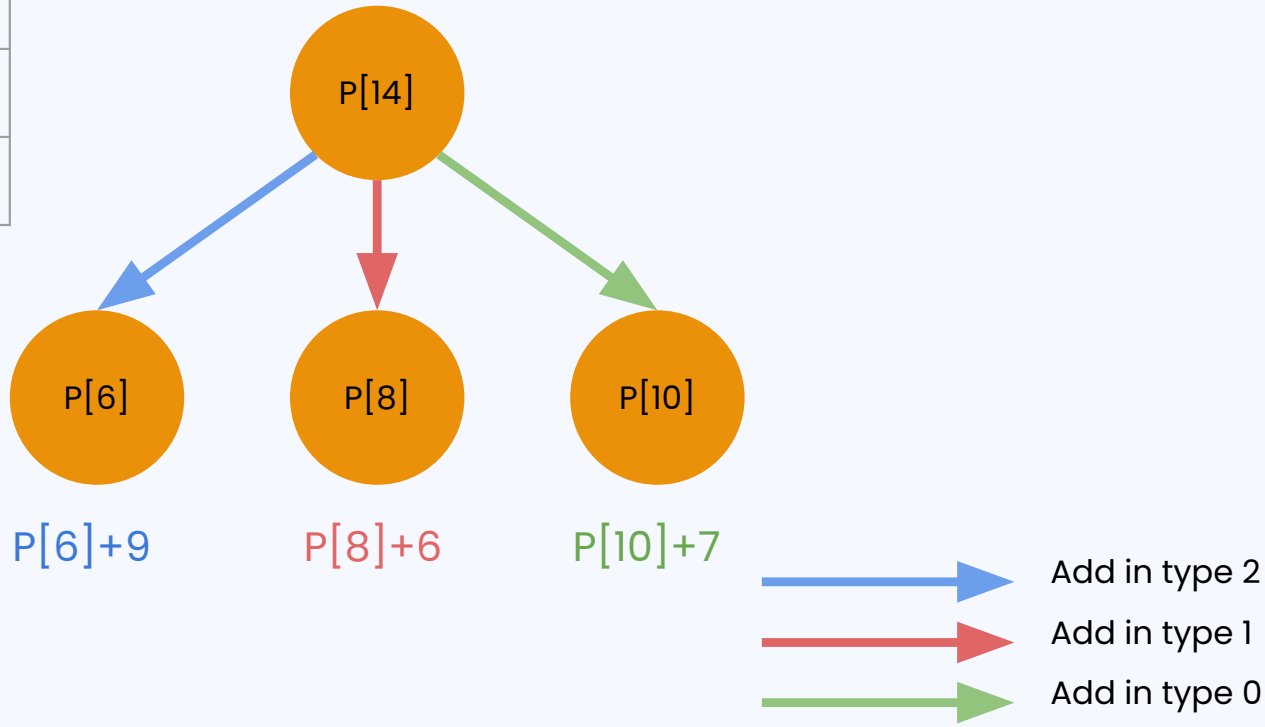
Idea: Consider the last item added, choose to add in the type of item that gives the maximum profit, profit of choosing a type u is given by $P(C-w_u)+p_u$

$P(C) = \max(P(C-w_0)+p_0, P(C-w_1)+p_1 , P(C-w_2)+p_2 , \dots , P(C-w_{n-1})+p_{n-1})$

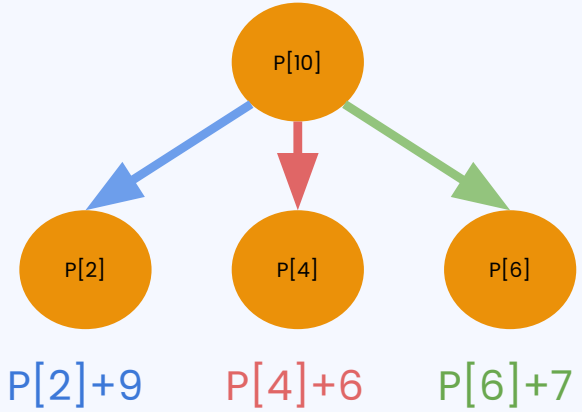
where w_n : weight of type n, p_n: profit of type n, P(i): maximum profit of knapsack with capacity, i

Subproblem graph for P(14)

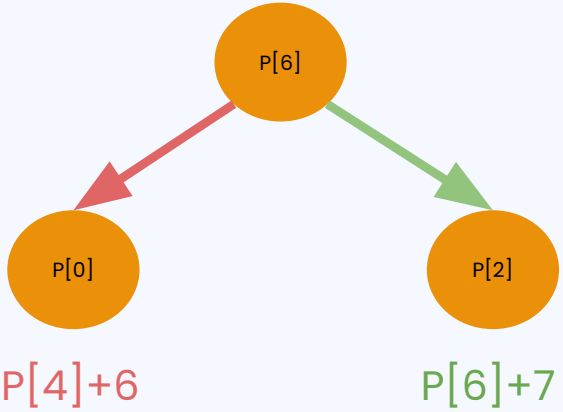
	0	1	2
w_i	4	6	8
p_i	7	6	9

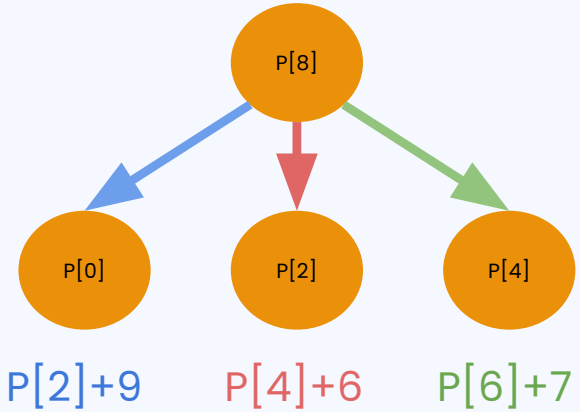


Subproblem graph for P(14)



	0	1	2
w_i	4	6	8
p_i	7	6	9

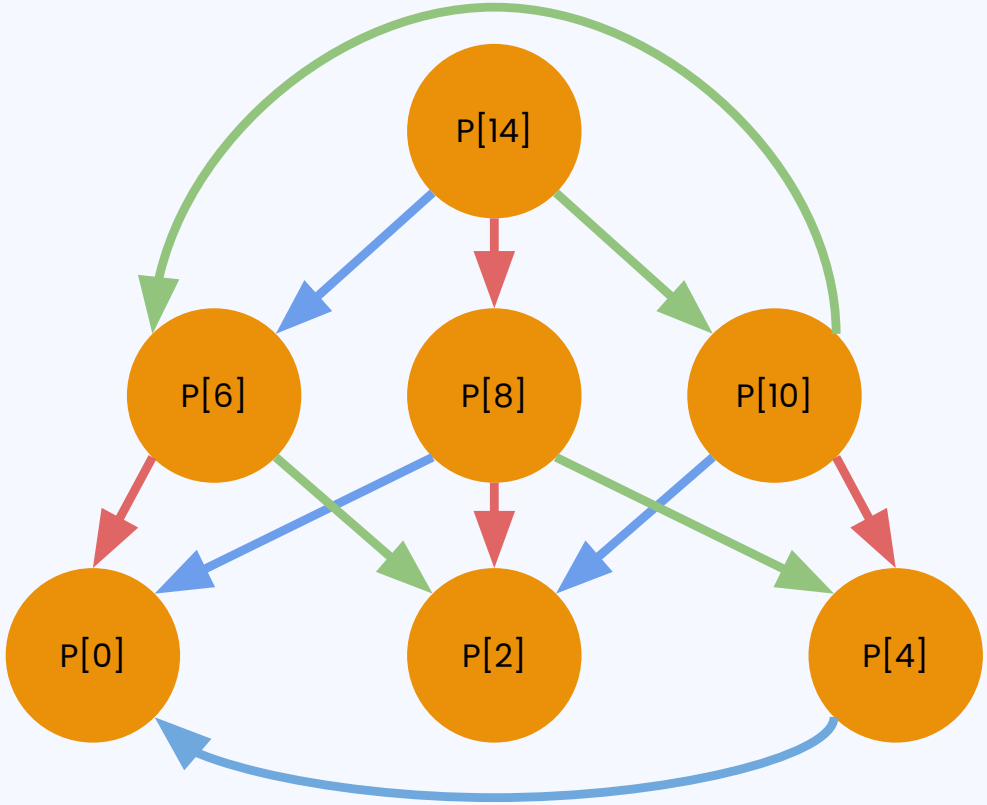




...

Subproblem graph for P(14)

	0	1	2
w_i	4	6	8
p_i	7	6	9



The dynamic programming algorithm

- Approach: bottom-up (from smallest problem to be built up to bigger ones)
- Dividing problem into subproblems will encounter overlapping ones
- Memoization is used (each subproblem is stored in a dictionary, to be referred later, so that each subproblem is only solved once)
- Optimization problem: all subproblems satisfy Principle of Optimality

Algorithm

```
int knapsack(int [] weight, int[] profit, int n, int C){
// P[i] is the max profit with weight i (initialized at 0)
    for i=0 to C+1{
        for j=0 to n{
            if ( i>=weight[j] and P[ i-w[j] ] + profit[j] > P[i] ){
                P[i] = P[ i-w[j] ] + profit[j];
            }
        }
    }
    return P[C];
}
```

Java Program

```java
public class Knapsack {
    private int[] w;
    private int[] p;
    private int n;
    private int[] profit;
    private int C;

    public Knapsack(int[] w, int[] p, int n, int C){
        this.w = w;
        this.p = p;
        this.n = n;
        this.C=C;
        this. profit = new int[C+1];
        for (int i=0; i<n; i++){ profit[i]=0;}
    }

    public int KnapsackDP(){
        for (int i = 0; i < C+1; i++) {
            for (int j = 0; j < n; j++) {
                if ( i>= w[j] && profit[i-w[j]] + p[j] > profit[i] ){
                    profit[i] = profit[i-w[j]] + p[j];
                }
            }
        }
        return profit[C];
    }

    public void printProfit(){
        System.out.println();
        for (int i = 0; i < profit.length; i++){
            System.out.println("profit[" + i + "] \t: " + profit[i] );
        }
        System.out.println("The maximum profit for this knapsack is " + profit[C]);
    }
}
```

```java
import java.util.*;

public class implementationClass {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner (System.in);
        System.out.print(s:"How many types of items are there?: ");
        int n = sc.nextInt();
        int [] w = new int[n];
        int [] p = new int[n];

        System.out.print(s:"What is the maximum capacity of the Knapsack?: ");
        int C  = sc.nextInt();

        for (int i = 0; i < n; i++){
            System.out.print("Insert weight for item " + i + " (weight):");
            w[i] = sc.nextInt();
            System.out.print("Insert profit for item " + i + " (profit):");
            p[i] = sc.nextInt();
        }

        Knapsack kn = new Knapsack (w,p,n,C);
        kn.KnapsackDP();
        kn.printProfit();

        sc.close();
    }
}
```

Java Program

```java
import java.util.*;

public class implementationClass {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner (System.in);
        System.out.print(s:"How many types of items are there?: ");
        int n = sc.nextInt();
        int [] w = new int[n];
        int [] p = new int[n];

        System.out.print(s:"What is the maximum capacity of the Knapsack?: ");
        int C  = sc.nextInt();

        for (int i = 0; i < n; i++){
            System.out.print("Insert weight for item " + i + " (weight):");
            w[i] = sc.nextInt();
            System.out.print("Insert profit for item " + i + " (profit):");
            p[i] = sc.nextInt();
        }

        Knapsack kn = new Knapsack (w,p,n,C);
        kn.KnapsackDP();
        kn.printProfit();

        sc.close();
    }
}
```

```java
    public Knapsack(int[] w, int[] p, int n, int C){
        this.w = w;
        this.p = p;
        this.n = n;
        this.C=C;
        this. profit = new int[C+1];
        for (int i=0; i<n; i++){ profit[i]=0;}
    }
```

```java
public int KnapsackDP(){
    for (int i = 0; i < C+1; i++) {
        for (int j = 0; j < n; j++) {
            if ( i>= w[j] && profit[i-w[j]] + p[j] > profit[i] ){
                profit[i] = profit[i-w[j]] + p[j];
            }
        }
    }
    return profit[C];
}
```

Problem 1:

We want to solve the unbounded knapsack problem where n = 3, and weights and profits given on the right

	0	1	2
w_i	4	6	8
p_i	7	6	9

Solve:
P(14)

	0	1	2
w_i	4	6	8
p_i	7	6	9

```
for i=0 to C{
        for j=0 to n-1{
                if ( i>=weight[j] and P[ i-w[j] ] + profit[j] > P[i] ){
                        P[i] = P[ i-w[j] ] + profit[j];
                }
        }
}
```

P(0)	P(1)	P(2)	P(3)	P(4)	P(5)	P(6)	P(7)	P(8)	P(9)	P(10)	P(11)	P(12)	P(13)	P(14)
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	0	1	2
w_i	4	6	8
p_i	7	6	9

```
for i=0 to C{
        for j=0 to n-1{
                if ( i>=weight[j] and P[ i-w[j] ] + profit[j] > P[i] ){
                        P[i] = P[ i-w[j] ] + profit[j];
                }
        }
}
```

P(0)	P(1)	P(2)	P(3)	P(4)	P(5)	P(6)	P(7)	P(8)	P(9)	P(10)	P(11)	P(12)	P(13)	P(14)
0	0	0	0	7	7	7	7	0	0	0	0	0	0	0

	0	1	2
w_i	4	6	8
p_i	7	6	9

```
for i=0 to C{
        for j=0 to n-1{
                if ( i>=weight[j] and P[ i-w[j] ] + profit[j] > P[i] ){
                        P[i] = P[ i-w[j] ] + profit[j];
                }
        }
}
```

P(0)	P(1)	P(2)	P(3)	P(4)	P(5)	P(6)	P(7)	P(8)	P(9)	P(10)	P(11)	P(12)	P(13)	P(14)
0	0	0	0	7	7	7	0	0	0	0	0	0	0	0

	0	1	2
w_i	4	6	8
p_i	7	6	9

```
for i=0 to C{
        for j=0 to n-1{
                if ( i>=weight[j] and P[ i-w[j] ] + profit[j] > P[i] ){
                        P[i] = P[ i-w[j] ] + profit[j];
                }
        }
}
```

P(0)	P(1)	P(2)	P(3)	P(4)	P(5)	P(6)	P(7)	P(8)	P(9)	P(10)	P(11)	P(12)	P(13)	P(14)
0	0	0	0	7	7	7	7	14	0	0	0	0	0	0

	0	1	2
w_i	4	6	8
p_i	7	6	9

```
for i=0 to C{
        for j=0 to n-1{
                if ( i>=weight[j] and P[ i-w[j] ] + profit[j] > P[i] ){
                        P[i] = P[ i-w[j] ] + profit[j];
                }
        }
}
```

P(0)	P(1)	P(2)	P(3)	P(4)	P(5)	P(6)	P(7)	P(8)	P(9)	P(10)	P(11)	P(12)	P(13)	P(14)
0	0	0	0	7	7	7	7	14	0	0	0	0	0	0

	0	1	2
w_i	4	6	8
p_i	7	6	9

```
for i=0 to C{
        for j=0 to n-1{
                if ( i>=weight[j] and P[ i-w[j] ] + profit[j] > P[i] ){
                        P[i] = P[ i-w[j] ] + profit[j];
                }
        }
}
```

P(0)	P(1)	P(2)	P(3)	P(4)	P(5)	P(6)	P(7)	P(8)	P(9)	P(10)	P(11)	P(12)	P(13)	P(14)
0	0	0	0	7	7	7	7	14	14	14	14	21	0	0

	0	1	2
w_i	4	6	8
p_i	7	6	9

```
for i=0 to C{
        for j=0 to n-1{
                if ( i>=weight[j] and P[ i-w[j] ] + profit[j] > P[i] ){
                        P[i] = P[ i-w[j] ] + profit[j];
                }
        }
}
```

P(0)	P(1)	P(2)	P(3)	P(4)	P(5)	P(6)	P(7)	P(8)	P(9)	P(10)	P(11)	P(12)	P(13)	P(14)
0	0	0	0	7	7	7	7	14	14	14	14	21	0	0

	0	1	2
w_i	4	6	8
p_i	7	6	9

```
for i=0 to C{
        for j=0 to n-1{
                if ( i>=weight[j] and P[ i-w[j] ] + profit[j] > P[i] ){
                        P[i] = P[ i-w[j] ] + profit[j];
                }
        }
}
```

P(0)	P(1)	P(2)	P(3)	P(4)	P(5)	P(6)	P(7)	P(8)	P(9)	P(10)	P(11)	P(12)	P(13)	P(14)
0	0	0	0	7	7	7	7	14	14	14	14	21	0	0

	0	1	2
w_i	4	6	8
p_i	7	6	9

```
for i=0 to C{
        for j=0 to n-1{
                if ( i>=weight[j] and P[ i-w[j] ] + profit[j] > P[i] ){
                        P[i] = P[ i-w[j] ] + profit[j];
                }
        }
}
```

P(0)	P(1)	P(2)	P(3)	P(4)	P(5)	P(6)	P(7)	P(8)	P(9)	P(10)	P(11)	P(12)	P(13)	P(14)
0	0	0	0	7	7	7	7	14	14	14	14	21	21	21

Output of P(14) with weights and profits given below

	0	1	2
w_i	4	6	8
p_i	7	6	9

```
How many types of items are there?: 3
What is the maximum capacity of the Knapsack?: 14
Insert weight for item 0 (weight):4
Insert profit for item 0 (profit):7
Insert weight for item 1 (weight):6
Insert profit for item 1 (profit):6
Insert weight for item 2 (weight):8
Insert profit for item 2 (profit):9

profit[0]       : 0
profit[1]       : 0
profit[2]       : 0
profit[3]       : 0
profit[4]       : 7
profit[5]       : 7
profit[6]       : 7
profit[7]       : 7
profit[8]       : 14
profit[9]       : 14
profit[10]      : 14
profit[11]      : 14
profit[12]      : 21
profit[13]      : 21
profit[14]      : 21
The maximum profit for this knapsack is 21
```

Problem 2:

We want to solve the unbounded knapsack problem where n = 3, and weights and profits given on the right

	0	1	2
w_i	5	6	8
p_i	7	6	9

Solve:
P(14)

	0	1	2
w_i	5	6	8
p_i	7	6	9

```
for i=0 to C{
        for j=0 to n-1{
                if ( i>=weight[j] and P[ i-w[j] ] + profit[j] > P[i] ){
                        P[i] = P[ i-w[j] ] + profit[j];
                }
        }
}
```

P(0)	P(1)	P(2)	P(3)	P(4)	P(5)	P(6)	P(7)	P(8)	P(9)	P(10)	P(11)	P(12)	P(13)	P(14)
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	0	1	2
w_i	5	6	8
p_i	7	6	9

```
for i=0 to C{
        for j=0 to n-1{
                if ( i>=weight[j] and P[ i-w[j] ] + profit[j] > P[i] ){
                        P[i] = P[ i-w[j] ] + profit[j];
                }
        }
}
```

P(0)	P(1)	P(2)	P(3)	P(4)	P(5)	P(6)	P(7)	P(8)	P(9)	P(10)	P(11)	P(12)	P(13)	P(14)
0	0	0	0	0	7	7	0	0	0	0	0	0	0	0

	0	1	2
w_i	5	6	8
p_i	7	6	9

```
for i=0 to C{
        for j=0 to n-1{
                if ( i>=weight[j] and P[ i-w[j] ] + profit[j] > P[i] ){
                        P[i] = P[ i-w[j] ] + profit[j];
                }
        }
}
```

P(0)	P(1)	P(2)	P(3)	P(4)	P(5)	P(6)	P(7)	P(8)	P(9)	P(10)	P(11)	P(12)	P(13)	P(14)
0	0	0	0	0	7	7	0	0	0	0	0	0	0	0

	0	1	2
w_i	5	6	8
p_i	7	6	9

```
for i=0 to C{
        for j=0 to n-1{
                if ( i>=weight[j] and P[ i-w[j] ] + profit[j] > P[i] ){
                        P[i] = P[ i-w[j] ] + profit[j];
                }
        }
}
```

P(0)	P(1)	P(2)	P(3)	P(4)	P(5)	P(6)	P(7)	P(8)	P(9)	P(10)	P(11)	P(12)	P(13)	P(14)
0	0	0	0	0	7	7	7	9	9	0	0	0	0	0

	0	1	2
w_i	5	6	8
p_i	7	6	9

```
for i=0 to C{
        for j=0 to n-1{
                if ( i>=weight[j] and P[ i-w[j] ] + profit[j] > P[i] ){
                        P[i] = P[ i-w[j] ] + profit[j];
                }
        }
}
```

P(0)	P(1)	P(2)	P(3)	P(4)	P(5)	P(6)	P(7)	P(8)	P(9)	P(10)	P(11)	P(12)	P(13)	P(14)
0	0	0	0	0	7	7	7	9	9	14	14	14	16	16

Output of P(14) with weights and profits given below

	0	1	2
w_i	5	6	8
p_i	7	6	9

```
How many types of items are there?: 3
What is the maximum capacity of the Knapsack?: 14
Insert weight for item 0 (weight):5
Insert profit for item 0 (profit):7
Insert weight for item 1 (weight):6
Insert profit for item 1 (profit):6
Insert weight for item 2 (weight):8
Insert profit for item 2 (profit):9

profit[0]       : 0
profit[1]       : 0
profit[2]       : 0
profit[3]       : 0
profit[4]       : 0
profit[5]       : 7
profit[6]       : 7
profit[7]       : 7
profit[8]       : 9
profit[9]       : 9
profit[10]      : 14
profit[11]      : 14
profit[12]      : 14
profit[13]      : 16
profit[14]      : 16
The maximum profit for this knapsack is 16
```